

Práctica 3

Monitores

1. Se dispone de un puente por el cual puede pasar un solo auto a la vez. Un auto pide permiso para pasar por el puente, cruza por el mismo y luego sigue su camino.

A) No, el código no funciona correctamente por el while que aparece en el procedimiento entrarPuente(). El problema con controlar el acceso al puente mediante una sentencia while es que podría llegar a pasar que el proceso demorado, dentro de la cola, saliera más tarde de lo que debía haber salido o quizás incluso nunca llegue a salir.

Esto podría ocurrir si hay un proceso dormido en la cola, y otro proceso que se encuentra saliendo del puente, y cuando este proceso saliente hace el signal(cola) el proceso se despierte pero igual un tercer proceso que se encontraba compitiendo por el acceso al monitor llegue antes e incremente la variable cant, en cuyo caso, una vez el tercer proceso salió del procedure entrarPuente(), este proceso ya despertado volvería a hacer otra iteración del while (debido a que cant ES mayor a 0) y se volvería a dormir.

B) Si, de la siguiente manera:

```
MONITOR Puente
    cond cola;
    int cant = 0;

    procedure cruzarPuente()
        // Realizar acción de cruzar puente
    end;
END MONITOR;

PROCESS Auto [a:1..M]
    Puente.cruzarPuente();
END PROCESS;
```

Por la implementación del monitor, aquí no habría conflictos con el acceso al recurso ya que el monitor realiza exclusión mutua de forma implícita. Cada auto simplemente tendría que ejecutar el procedimiento cruzarPuente() cuando tenga la oportunidad.

- C) No, ya que podría darse que un proceso se vuelva a dormir antes de poder salir de la sentencia while. Para que si se respete el orden de llegada, el código debería reescribirse tal que así:

```
MONITOR Puente
    cond cola;
    int cant = 0;

    procedure entrarPuente()
        if (cant > 0) wait(col);
        cant = cant + 1;
    end;

    procedure salirPuente()
        cant = cant - 1;
        signal(col);
    end;
END MONITOR;

PROCESS Auto [a:1..M]
    Puente.entrarPuente();
    "El auto cruza el puente"
    Puente.salirPuente();
END PROCESS;
```

2. Existen N procesos que deben leer información de una base de datos, la cual es administrada por un motor que admite una cantidad limitada de consultas simultáneas.

- A) **Procesos:** Consulta → Representa una consulta a la BD
Recursos: La BD
Monitores: Motor → Tiene dos procedimientos donde se controla el acceso y la salida a la base de datos

```
B) MONITOR Motor
    int llegando = 0;
    int usando = 0;
    cond vc;

    procedure entrar()
        if (usando < 5)
            usando++;
        else
            llegando++; wait(vc);
        end;

    procedure salir()
```

```

        if (llegando > 0)
            llegando--; signal(vc);
        usando--;
    END MONITOR;

    PROCESS Consulta[1..N]
        Motor.entrar();
        // Hacer cosas en la BD
        Motor.salir();
    END PROCESS;

```

3. Existen N personas que deben fotocopiar un documento. La fotocopidora sólo puede ser usada por una persona a la vez. Analice el problema y defina qué procesos, recursos y monitores serán necesarios/convenientes, además de las posibles sincronizaciones requeridas para resolver el problema. Luego, resuelva considerando las siguientes situaciones:

A) MONITOR Mon_foto

```

    procedure usar()
        fotocopiar();
    end;
END MONITOR;

```

```

    PROCESS PERSONA[1..N]
        Mon_foto.usar();
    END PROCESS;

```

B) MONITOR Mon_foto

```

    int llegando = 0;
    bool usando = false;
    cond vc;

    procedure solicitar()
        if (usando)
            llegando++; wait(cv);
        else
            usando = true;
        end;

    procedure salir()
        if (llegando > 0)
            signal(vc); llegando--;
        else
            usando = false;
        end;
    end;

```

```
END MONITOR;
```

```
PROCESS PERSONA[1..N]  
    Mon_foto.solicitar();  
    Fotocopiar();  
    Mon_foto.salir();  
END PROCESS;
```

```
C) MONITOR Mon_foto  
    cond[] espera[N];  
    cola colaEdad;  
    bool usando = false;  
    int llegando = 0;  
  
    procedure solicitar(idP, edad: int)  
        if (usando)  
            llegando++; wait(cv);  
            colaEdad.push(idP, edad);  
            wait(espera[idP]);  
        else  
            usando = true;  
        end;  
  
    procedure salir()  
        if (llegando > 0)  
            int aux = colaEdad.pop();  
            signal(espera[aux]);  
            llegando--;  
        else  
            usando = false;  
        end;  
END MONITOR;
```

```
PROCESS PERSONA[Id: 1..N]  
    int edad = ...;  
    Mon_foto.solicitar(Id, edad);  
    Fotocopiar();  
    Mon_foto.salir();  
END PROCESS;
```

```
D) MONITOR Mon_foto  
    cond[] espera[N];  
    int indice = 1;  
  
    procedure entrar(id: int)  
        if (id == indice)  
            wait(espera[id])  
        end;  
end;
```

```

        procedure salir()
            indice++;
            signal(espera[indice]);
        end;
    END MONITOR;

```

```

PROCESS PERSONA[Id: 1..N]
    Mon_foto.entrar(id);
    Fotocopiar();
    Mon_foto.salir();
END PROCESS;

```

```

E) MONITOR Mon_foto
    int llegando 0;
    cond vc;
    bool usando = false;
    bool llego_per = false;
    cond despertar;

    procedure entrar()
        llegando++;
        wait(usar_imp);
    end;

    procedure salir()
        signal(liberar_imp);
        llegando--;
    end;

    procedure conceder()
        if (llegando > 0)
            signal(usar_imp)
            wait(liberar_imp)
        end;
    END MONITOR;

PROCESS PERSONA[Id: 1..N]
    Mon_foto.entrar();
    Fotocopiar();
    Mon_foto.salir();
END PROCESS;

PROCESS EMPLEADO
    for i:=1 to N do
        Mon_foto.controlar();
    end;
END PROCESS;

```

F) MONITOR Mon_foto

```
cola<Integer> cola_id_fotoc[10];
cond solicitud_fotoc;
cond usar_fotoc;
cond fotoc_disponible;
int personas_esperando = 0;
int personas_usando = 0;

procedure entrar(id_fotoc: out int)
    personas_esperando++;
    if (personas_usando > 10)
        wait(fotoc_disponible);
    if (personas_esperando > 0)
        while (personas_usando < 10 && personas_esperando > 0)
            signal(fotoc_disponible);
        if (personas_usando > 9)
            wait(fotoc_disponible)
    personas_usando++;
    personas_esperando--; → Por ahí está de más esto

    signal(solicitud_fotoc);
    wait(usar_fotoc);
    id_fotoc = cola_id_fotoc.pop();
end;

procedure salir(id_fotoc: int)
    cola_id_fotoc.push(id_fotoc);
    personas_usando--;
    colaIntFoto.push(Id);
    while (personas_usando < 10 && personas_esperando > 0)
        signal(fotoc_disponible);
end;

procedure otorgar_fotoc()
    wait(solicitud_fotoc);
    signal(usar_fotoc);
    // Personas usando nunca deberían superar los 10
    while (personas_usando < 10 && personas_esperando > 0)
        signal(fotoc_disponible);
end;
END MONITOR;

PROCESS PERSONA[Id: 1..N]
    int aux;
    Mon_foto.entrar(aux);
    Fotocopiar(aux);
    Mon_foto.salir(aux);
END PROCESS;
```

```

PROCESS EMPLEADO
  for i:=1 to N do
    Mon_foto.otorgar_fotoc();
  end;
END PROCESS;

```

CONSULTAR

Situaciones posibles:

Supongamos que entra una persona sola --> Incrementa el contador de espera --> No entra en el IF --> Incrementa el contador del IF y decrementa el primero --> Le avisa al empleado que quiere utilizar la fotocopidora, y se queda esperando en una cola --> En ese momento, el empleado se despierta de la cola y despierta a algún proceso que esté dormido en la cola (en este caso, el único que hay) --> El empleado se fija si hay 10 personas utilizando la impresora, y si hay alguien esperando (en este caso no hay nadie por lo que no entra) --> La persona obtiene un id de fotocopidora y termina el procedure --> Ya en este punto no interesa lo que haga el empleado, porque se va a quedar esperando a que haya otra persona --> La persona fotocopia, termina de utilizar su fotocopidora, avisa que esta está libre, se decrementa el contador del IF, y devuelve el id de su fotocopidora a la cola

Supongamos que entran 11 personas → Supongamos que llegan los 10 primeros, todos incrementan el contador de espera, incrementan el contador del IF, decrementan el contador de espera, y esperan a que el empleado los despierte y reciban un ID de impresora --> Si en este momento llegara a venir un onceavo proceso, entraría por el primer IF y se dormiría --> Supongamos que entra un doceavo proceso y ya hay una fotocopidora libre (personas_usando = 9), el doceavo proceso incrementaría el contador de espera (personas_esperando = 2), no entraría en el primer IF (personas_usando < 10), incrementaría el contador del IF y decrementaría el de esperando (personas_usando = 10, personas_esperando = 1), acto seguido, le avisaría al empleado que quiere fotocopiar, y el empleado le daría permiso y recibiría un id --> El empleado después de todo esto se fijaría si puede despertar a algún proceso, y se da cuenta de que no al estar (personas_usando = 10) [¿DEMORA INNECESARIA? → Consultar]

Se me ocurre la solución azul, pero tampoco creo que esté bien

- Existen N vehículos que deben pasar por un puente de acuerdo con el orden de llegada. Considere que el puente no soporta más de 50000kg y que cada vehículo cuenta con su propio peso (ningún vehículo supera el peso soportado por el puente).

MONITOR Puente

```

int peso_actual = 0;
int[] pesos = ([N] 0);
cond cruzar;
int cant = 0;

procedure pasar(peso: int)
    if (peso_actual + peso > 50.000 && cant == 0)
        wait(cruzar);
        pesos[cant] = peso;
        cant++;
        peso_actual += peso;
    end;

procedure salir(peso: int)
    peso_actual -= peso;
    while (peso_actual + pesos[cant] <= 50.000)
        signal(cruzar);
        cant--;
    end;
END MONITOR

PROCESS AUTO[1..N]
    int peso = Math.random(1, 50.000);
    Puente.pasar(peso);
    // Cruzar puente
    Puente.salir(peso);
END PROCESS;

```

5. En un corralón de materiales se deben atender a N clientes de acuerdo con el orden de llegada. Cuando un cliente es llamado para ser atendido, entrega una lista con los productos que comprará, y espera a que alguno de los empleados le entregue el comprobante de la compra realizada.

- A) A
- B) Con M siendo el número de empleados

```

MONITOR Corralon
    cola cola_ids_empl_disp = cola<Integer>([M]);
    Compra[] compra_por_id_empl = ([M] null);
    Recibo[] recibo_por_id_empl = ([M] null);
    cond[] vc_id_empl = ([M] cond)
    cond[] vc_id_cli = ([M] cond)
    int[] id_cliente_correspondiente = ([M] 0);
    cond cola_general;
    bool pasar = true;
    personas_atendidas = 0;

```

```

procedure entrar(id: int, compra[] cli_compra, id_empl: out int)
    if (!pasar)
        wait cola_general
        personas_atendidas++;
        if (personas_atendidas > M)
            pasar = false
        id_empl = cola_ids_empl_dsp.pop();
        id_cliente_correspondiente[id_empl] = id;
        compra_por_id_empl[id_empl] = cli_compra;
        signal(vc_id_empl[id_empl]);
    end;

```

```

procedure salir(id: int, recibo: out recibo, id_empl: int)
    wait(vc_id_cli[id]);
    aux_recibo = recibo_por_id_empl[id_empl];
    recibo = aux_recibo;
    cola_ids_empl_disp.push(id_empl);
    personas_atendidas--;
    if (personas_atendidas == 0)
        pasar = true;
    signal(cola_general);
end;

```

```

procedure atender(id_empl: int)
    wait(vc_id_empleado[id_empl]);
    aux_id = id_cliente_correspondiente[id_empl];
    aux_compra = compra_por_id_empl[aux_id];
    aux_recibo = procesar(aux_compra);
    recibo_por_id_empl[aux_id];
    signal(vc_id_cli[aux_id]);
end;

```

END MONITOR

```

PROCESS CLIENTE[Id: 1..N]
    compra[] cli_compra;
    int id_empl;
    Corralon.entrar(id, cli_compra, id_empl);
    recibo aux_recibo;
    Corralon.salir(id, aux_recibo, id_empl);
END PROCESS

```

```

PROCESS EMPLEADO[Id: 1..M]
    for i:=1 to N do
        Corralon.atender(Id);
    end;
END PROCESS

```

C) Con M siendo el número de empleados

MONITOR Corralon

```
cola cola_ids_empl_disp = cola<Integer>([M]);
Compra[] compra_por_id_empl = ([M] null);
Recibo[] recibo_por_id_empl = ([M] null);
cond[] vc_id_empl = ([M] cond)
cond[] vc_id_cli = ([M] cond)
int[] id_cliente_correspondiente = ([M] 0);
cond cola_general;
bool pasar = true;
int personas_atendidas = 0;
int ya_compraron = 0;
cond despertar_empl_esp;
```

```
procedure entrar(id: int, compra[] cli_compra, id_empl: out int)
  if (!pasar)
    wait(cola_general)
    personas_atendidas++;
    if (personas_atendidas > M)
      pasar = false
    id_empl = cola_ids_empl_disp.pop();
    id_cliente_correspondiente[id_empl] = id;
    compra_por_id_empl[id_empl] = cli_compra;
    signal(vc_id_empl[id_empl]);
end;
```

```
procedure salir(id: int, recibo: out recibo, id_empl: int)
  wait(vc_id_cli[id]);
  aux_recibo = recibo_por_id_empl[id_empl];
  recibo = aux_recibo;
  ya_compraron++;
  cola_ids_empl_disp.push(id_empl);
  personas_atendidas--;
  if (personas_atendidas == 0)
    pasar = true;
  signal(cola_general);

  if (ya_compraron == N)
    signal(despertar_empl_esp)
end;
```

```
procedure atender(id_empl: int)
  wait(vc_id_empleado[id_empl]);
  if (ya_compraron < N)
    aux_id = id_cliente_correspondiente[id_empl];
    aux_compra = compra_por_id_empl[aux_id];
    aux_recibo = procesar(aux_compra);
```

```

        recibo_por_id_empl[aux_id];
        signal(vc_id_cli[aux_id]);

    end;

    procedure controlar_horario();
        wait(despertar_empl_esp);
        for i:=1 to N do
            signal(vc_id_empl[i]);
        end;
    END MONITOR

PROCESS CLIENTE[Id: 1..N]
    compra[] cli_compra;
    int id_empl;
    Corralon.entrar(id, cli_compra, id_empl);
    recibo aux_recibo;
    Corralon.salir(id, aux_recibo, id_empl);
END PROCESS

PROCESS EMPLEADO[Id: 1..M]
    while (true)
        Corralon.atender(Id);
    END PROCESS

PROCESS EMPLEADO_ESP
    Corralon.controlar_horario();
END PROCESS

```

6. Existe una comisión de 50 alumnos que deben realizar tareas de a pares, las cuales son corregidas por un JTP. Cuando los alumnos llegan, forman una fila. Una vez que están todos en fila, el JTP les asigna un número de grupo a cada uno. Para ello, suponga que existe una función AsignarNroGrupo() que retorna un número “aleatorio” del 1 al 25. Cuando un alumno ha recibido su número de grupo, comienza a realizar su tarea. Al terminarla, el alumno le avisa al JTP y espera por su nota. Cuando los dos alumnos del grupo completaron la tarea, el JTP les asigna un puntaje (el primer grupo en terminar tendrá como nota 25, el segundo 24, y así sucesivamente hasta el último que tendrá nota 1). Nota: el JTP no guarda el número de grupo que le asigna a cada alumno.

```

PROCESS Alumno[1..50] {
    Comision.alumno_llegar()
    int num_grupo;
    Comision.alumno_recibir_num_grupo(num_grupo);
    // Realizar tarea
    Comision.alumno_avisar_jtp(num_grupo);
}

PROCESS JTP {
    Comision.jtp_llegar();

    for i:=1 to 50 do
        Comision.jtp_asignar_numero();
    for i:= 1 to 50 do
        Comision.jtp_controlar_alumnos();
}

MONITOR Comision {
    int barrera = 0; vc alumno_barrera; vc jtp_esperando_fila;
    bool fila_armada = false;
    int cont_alum_esperando_nota = 0; vc alum_esperando_nota;
    bool asig_nota_ocupado = false;
    bool alum_llego_para_recibir_nota; vc jtp_esperar_alum_para_nota;
    vc alum_esperar_para_nota; int cont_alum_esperando_nota;

    int nota = 25; int[] notas_grupos = ([25] 0); vc[] alum_esperar = ([25]);
    int[] cant_terminaron_grupo = ([25] 0); bool jtp_dormido = false;
    cola<Integer> cola_alum_terminaron; vc jtp_dormir_tarea;

    procedure alumno_llegar() {
        barrera++;
    }
}

```

```

        if (barrera < 50)
            wait(alumno_barrera);
        else
            signal(jtp_esperando_fila);
            fila_armada = true;
            wait(alumno_barrera); → Supuestamente por la EM se debería
ejecutar primero este
    }

```

```

procedure alumno_recibir_num_grupo(numgrp: out int) {
    if (asig_nota_ocupado)
        cont_alum_esperando_nota++;
        wait(alum_esperando_nota);
    asig_nota_ocupado = true;
    signal(jtp_esperar_alum_para_nota);
    alum_llego_para_recibir_nota = true;
    wait(alum_esperar_para_nota);
    numgrp = num_grupo;
}

```

```

procedure alumno_avisar_jtp(numgrp: int) {
    cola_alum_terminaron.push(numgrp);
    if (jtp_dormido)
        signal(jtp_dormir_tarea);
    wait(alum_esperar[numgrp]);
}

```

```

procedure jtp_llegar() {
    if (not fila_armada) wait(jtp_esperando_fila);
    signal_all(alumno_barrera); → Despertar a todos en la fila
}

```

```

procedure jtp_asignar_numero() {
    if (not alum_llego_para_recibir_nota) wait(jtp_esperar_alum_para_nota) →
Sincronización

```

```

    num_grupo = AsignarNroGrupo(); → Pasar número por var. permanente
    signal(alum_esperar_para_nota)
    alum_llego_para_recibir_nota = false;
    if (cont_alum_esperando_nota > 0)
        signal(alum_esperando_nota);
    asig_nota_ocupado = false;
    cont_alum_esperando_nota--;
}

```

```

procedure jtp_asignar_puntos() {
    if (empty(cola_alum_terminaron))
        jtp_dormido = true;
}

```

```

        wait(jtp_dormir_tarea);
    else
        int num_grupo = cola_alum_terminaron.pop();
        cant_terminaron_grupo[num_grupo]++;
        if (cant_terminaron_grupo[num_grupo] == 2)
            notas_grupos[num_grupo] = nota;
            nota--;
            signal_all(alum_esperar[num_grupo]);
    }
}

```

7. Se debe simular una maratón con C corredores donde en la llegada hay UNA máquina expendedoras de agua con capacidad para 20 botellas. Además, existe un repositor encargado de reponer las botellas de la máquina. Cuando los C corredores han llegado al inicio comienzala carrera. Cuando un corredor termina la carrera se dirigen a la máquina expendedora, espera su turno (respetando el orden de llegada), saca una botella y se retira. Si encuentra la máquina sin botellas, le avisa al repositor para que cargue nuevamente la máquina con 20 botellas; espera a que se haga la recarga; saca una botella y se retira. Nota: mientras se reponen las botellas se debe permitir que otros corredores se encolen.

[CORREGIDO]

```

PROCESS Corredor [1..C] {
    botella aux_botella;
    Carrera.correr();
    // Correr
    Fila.encolarse();
    MaqExp.sacar_botella(aux_botella);
    Fila.retirarse(); → ¿Este sería responsabilidad de Fila?
}

```

```

PROCESS Repositor {
    while (true) {
        MaqExp.repositor_esperar();
    }
}

```

```

MONITOR Carrera {
    vc barrera;
    int cont_barrera = 0;

    procedure correr() {

```

```

        cont_barrera++;
        if (cont_barrera < C)
            wait(barrera)
        else
            signal_all(barrera);
        → “Cuando los C corredores han llegado al inicio comienza la carrera”
    }
}

```

MONITOR Fila {

```

bool maquina_ocupada = false;
vc maquina_ocupada;

```

procedure encolarse() {

→ En este procedure se controla como se va usando la máquina

```

    if (maquina_ocupada)
        cont_corredor_espera++;
        wait(esperando_maquina);

```

```

        maquina_ocupada = true;

```

```

    }

```

procedure retirarse() {

```

    if (cont_corredor_espera > 0)
        cont_corredor_espera--;
        signal(esperando_maquina);
    else
        maquina_ocupada = false;

```

```

    }

```

```

}

```

MONITOR MaqExp {

```

cola<Botella> cola_botellas = new Cola<Botella>();

```

procedure sacar_botella(botella: out botella) {

```

    if (empty(cola_botellas))
        signal(no_hay_botellas);
        wait(esperar_botellas);
        botella = cola_botellas.pop();

```

```

    }

```

```

procedure repositor_esperar() {
    if (not empty(cola_botellas)) wait(no_hay_botellas)
    for i:=1 to 20 do
        cola_botellas.push(botella);
    signal(esperar_botella)
}
}

```

8. En un entrenamiento de fútbol hay 20 jugadores que forman 4 equipos (cada jugador conoce el equipo al cual pertenece llamando a la función DarEquipo()). Cuando un equipo está listo (han llegado los 5 jugadores que lo componen), debe enfrentarse a otro equipo que también esté listo (los dos primeros equipos en juntarse juegan en la cancha 1, y los otros dos equipos juegan en la cancha 2). Una vez que el equipo conoce la cancha en la que juega, sus jugadores se dirigen a ella. Cuando los 10 jugadores del partido llegaron a la cancha comienza el partido, juegan durante 50 minutos, y al terminar todos los jugadores del partido se retiran (no es necesario que se esperen para salir).

[CORREGIDO]

```

PROCESS Jugador[1..20] {

    int num_equipo = DarEquipo();
    int num_cancha;

    EquipoBarrera[num_equipo].llegar();
    JuntarEquipos.recibir_cancha(num_equipo, num_cancha);
    Cancha[num_cancha].llegar();

}

```

```

MONITOR Equipos [0..3] {

    int cont_barrera = 0;
    int[] num_cancha_equipo = ([3] 0);
    vc barrera;

    procedure llegar(num_equipo: in int; num_cancha: out int) {
        → Barrera local del equipo
        cont_barrera++;
        if (cont_barrera < 5)
            wait(barrera);
        else

```

```

        JuntarEquipos.recibir_numero(num_cancha_equipo[num_equipo]);
        signal_all(barrera);
        num_cancha = num_cancha_equipo[num_equipo];
    }
}

MONITOR JuntarEquipos {
    int equipos_listos = 0;

    procedure recibir_cancha(num_cancha: out int) {
        num_cancha = equipos div 2 + 1;
        equipos++;
    }

MONITOR Cancha [1..2] {

    int cont_barrera = 0;
    vc barrera;

    procedure llegar() {
        → Barrera local de la cancha
        cont_barrera++;
        if (cont_barrera < 10)
            wait(barrera);
        else
            delay(50 minutos);
            signal_all(barrera);
    }
}
}

```

9. En un examen de la secundaria hay un preceptor y una profesora que deben tomar un examen escrito a 45 alumnos. El preceptor se encarga de darle el enunciado del examen a los alumnos cuando los 45 han llegado (es el mismo enunciado para todos). La profesora se encarga de ir corrigiendo los exámenes de acuerdo con el orden en que los alumnos van entregando. Cada alumno al llegar espera a que le den el enunciado, resuelve el examen, y al terminar lo deja para que la profesora lo corrija y le envíe la nota. Nota: maximizar la concurrencia; todos los procesos deben terminar su ejecución; suponga que la profesora tiene una función `corregirExamen` que recibe un examen y devuelve un entero con la nota.

[CORREGIDO]

```
PROCESS Alumno[Id: 1..45] {
    enunciado aux_enunciado;
    int nota;

    Aula.alumno_llegar(aux_enunciado); → Va a tener que haber una barrera
    examen aux_examen = resolver(aux_enunciado);
    Aula.alumno_entregar_examen(aux_examen, nota);
}
```

```
PROCESS Preceptor {
    enunciado aux_enunciado = cargar_enunciado();
    Aula.preceptor_llegar(aux_enunciado);
}
```

```
PROCESS Profesora {
    for (int i; i = 0; i < 45) {
        examen aux_examen;
        Aula.profesora_recibir_examen(aux_examen);
        int nota_examen = corregirExamen(aux_examen);
        Aula.profesora_entregar_nota(nota_examen);
    }
}
```

MONITOR Aula {

→ Entre el alumno y el preceptor
vc espera_preceptor_listo;
vc espera_alumno_lleguen;

→ Entre el alumno y la profesora
cola<Integer> cola_id;
vc espera_nota
int[] arreglo_notas = ([45] 0)

```
procedure alumno_llegar(enunciado: out enunciado) {
    → Barrera entre alumnos
    cont_alumnos_llegaron++;
    if (cont_alumnos_llegaron == 45)
        signal(espera_preceptor_listo)
    wait(espera_alumnos_lleguen);
    enunciado = enun;
}
```

```
procedure preceptor_llegar() {
    if (cont_alumnos_llegaron < 45)
        wait(espera_preceptor_listo);
    aux_enunciado = enun; → Pasaje
```

```

        signal_all(espera_alumno_lleguen); → Todos los alumnos reciben enunciado
    }

    procedure alumno_entregar_examen(id: in int; aux_examen: in examen; nota: out
int) {
        cola_id.push(id, aux_examen);
        signal(espera_profesora)
        wait(espera_alumno_examen)
        nota = arreglo_notas[id];
    }

    procedure profesora_recibir_examen(id: out int; aux_examen: out examen) {
        if (empty(cola_id))
            wait(espera_profesora)
        id, aux_examen = cola_id.pop();
    }

    procedure profesora_entregar_nota(id, nota_examen: in int) {
        arreglo_notas[id] = nota_examen;
        signal(espera_alumno_examen); → Despertar alumno
    }
}

```

CONSULTAR → Siento que el tema de la corrección podría ser manejado mas eficientemente. No sé si esté mal que cada alumno tenga que sincronizarse con la profesora para la corrección de su examen, o si hay que manejarlo con una cola para que sea más rápido.

→ Otra solución posible es hacer dos monitores, uno para la barrera y el enunciado, y otro para la profesora (!)

→ Cuando sean 45, despierten al preceptor y se duerman

10. En un parque hay un juego para ser usada por N personas de a una a la vez y de acuerdo al orden en que llegan para solicitar su uso.

Además, hay un empleado encargado de desinfectar el juego durante 10 minutos antes de que una persona lo use. Cada persona al llegar espera hasta que el empleado le avisa que puede usar el juego, lo usa por un tiempo y luego lo devuelve. Nota: suponga que la persona tiene una función Usar_juego que simula el uso del juego; y el empleado una función Desinfectar_Juego que simula su trabajo. Todos los procesos deben terminar su ejecución.

[CORREGIDO]

```

PROCESS Persona [1..N] {
    Juego.persona_esperar();
    Persona.Usar_juego();
    Juego.persona_salir();
}

```

```
}
```

```
PROCESS Empleado {  
    bool ultima_persona = false;  
    while (not ultima_persona) {  
        Empleado.Desinfectar_Juego();  
        Juego.empleado_esperar(ultima_persona);  
    }  
}
```

```
MONITOR Juego {
```

```
    bool juego_listo = true;  
    int cont_personas_esperando = 0;  
    int cont_personas_pasaron = 0;  
    vc espera_juego_listo;  
    vc despertar_empleado;
```

```
    procedure persona_esperar() {  
        → Las personas llegan y deben esperar a que el juego se desocupe  
        if (not juego_listo)  
            cont_personas_esperando++;  
            wait(espera_juego_listo)  
        else  
            juego_listo = false;  
    }
```

```
    procedure persona_salir() {  
        cont_personas_pasaron++;  
        signal(despertar_empleado);  
    }
```

```
    procedure empleado_esperar(ultima_persona: out bool) {  
        if (cont_personas_pasaron != N) wait(despertar_empleado)  
        ultima_persona = (cont_personas_pasaron < N);  
  
        if (cont_personas_esperando > 0)  
            cont_personas_esperando--;  
            signal(espera_juego_listo);  
        else  
            juego_listo = true;  
    }
```

```
}
```